영아 안전을 위한
능동형 콘센트 접근 차단 로봇 시스템

2391010 김휘서

프로젝트 배경 및 필요성

48.1%

전기 안전사고 피해자 중 **10세 미만 어린이** 비율

전기 안전사고는 다시 증가하는 추세에 있습니다. 나이가 확인된 안전사고의 48.1%가 10세 미만 어린이로 집계됐으며, 그 중 인지·신체 능력이 미발달한 영아가 특히 위험에 노출되어 있습니다.

기존 수동형 콘센트 커버는 영아가 스스로 제거할 수 있어 근본적인 해결책이 되지 못합니다. 본 연구는 **물리적 경로 차단**에 유의미한 결과를 보일 것으로 예상되는 영아를 대상으로 설계되었습니다.

- **로봇이 영아를 실시간 감지하여 위험한 콘센트 앞으로 자율 이동, 물리적으로 접근 차단**
- **인지·신체 능력 미발달 영아 — 물리적 경로 차단이 가장 효과적인 연령대**

System Architecture

RGB 카메라

로봇 탑재 카메라

YOLOv8

영아 실시간 탐지

outlet_selector

최근접 콘센트 계산
Actor 위치 수신

Nav2

NavigateToPose
자율 주행

로봇 이동

Blocking Point
도달 및 차단

시뮬레이션

Gazebo Fortress

미들웨어

ROS2 Humble

영아 탐지

YOLOv8n (CPU)

자율 주행

Nav2 + RPP Controller

시각화

RViz2

위치 추정

Odometry 기반

구현 방법

1. 영아 탐지 — yolo_node.py

- RGB 카메라 → YOLOv8n CPU 추론
- 바운딩 박스 면적 비율로 필터링
- 가장 큰 물체를 영아로 판단
- /baby_detected (Bool) 퍼블리시

```
for box in results[0].boxes:
    x1, y1, x2, y2 = box.xyxy[0].tolist()
    area = (x2 - x1) * (y2 - y1)
    ratio = area / (w * h)
    if 0.01 < ratio < 0.5 and area > max_area:
        max_area = area
        baby_box = box
```

2. 콘센트 선택 — outlet_selector.py

- 4개 콘센트 좌표 사전 등록
- ActorPosePublisher로 영아 위치 수신
- 유클리드 거리 기반 최근접 콘센트 선택
- 3초 미감지 → 네비게이션 취소
- 5초 미감지 → 제자리 회전 탐색

} baby_detected = True → 즉시 회전 중단, 새 목표 설정

```
def _pose_cb(self, msg: PoseStamped):
    if not self._baby_detected:
        return

    bx, by = msg.pose.position.x, msg.pose.position.y
    self._actor_pose = (bx, by)
    best = min(
        OUTLETS,
        key=lambda o: (bx - o['outlet'][0]) ** 2 + (by - o['outlet'][1]) ** 2,
    )

    if best['id'] == self._current_goal_id:
        return

    if self._navigating:
        self._cancel_current()

    self._current_goal_id = best['id']
    gx, gy, gyaw = best['blocking']
    self.get_logger().info(
        f'Baby ({bx:.2f},{by:.2f}) → outlet {best["id"]} → blocking ({gx},{gy})')
    self._send_goal(gx, gy, gyaw)
```

구현 방법

3. 자율 주행 — Nav2 (RPP)

- Regulated Pure Pursuit Controller
- 목표 방향 오차 >45° → 제자리 회전 선행
- Odometry 기반 로컬라이제이션
- 도착 후 영아 방향으로 회전 (atan2)
- 목표 변경 시 즉시 새 목표 전환

4. 영아 모델 — actor_world.sdf

- Gazebo Human Actor 사용
- 반경 5m 원형 경로 이동
- 콘센트 근처 waypoint에서 6초 정지
- 속도: 0.3 m/s (로봇보다 느리게)
- 콘센트 위치는 RViz Marker로만 표시

```
<!-- 원형 경로 8개 waypoint -->
<waypoint><x>5.0</x><y>0.0</y><pause>0</pause></waypoint>
<waypoint><x>4.5</x><y>4.5</y><pause>6</pause></waypoint>
<!-- 콘센트 근처에서 6초 정지 -->

<!-- 속도 -->
<velocity>0.3</velocity>
```

baby_safety.launch.py

노드 번호	노드	역할
1	yolo_detection.launch	Gazebo + 카메라 + YOLO
2	map_server	빈 맵 제공
3	static_transform_publisher	map→odom TF 발행
4-8	Nav2 스택	경로 계획 및 제어
9	lifecycle_manager	Nav2 노드 생명주기 관리
10	outlet_selector	콘센트 선택 및 Nav2 goal 전송
11	RViz	시각화

독립 blocking_arm

- 물리적 접근 차단을 구현하기 위해 별도 branch에서 blocking_arm 패키지를 추가
- 양팔 구조의 revolute joint 기반 arm 모델을 URDF/Xacro로 설계

→ 사람의 팔 구조에서 모티브

- Gazebo JointPositionController를 사용하여 각 관절 명령을 통해 arm을 제어

: f-전개. r-복원

- arm_controller_node.py에서 keyboard 입력 기반 fold / extend 동작
- main navigation loop와의 자동 연동은 난이도 및 이동로봇의 한계점을 고려

→ 최종 demo에서 탈락

- 6개 revolute joint를 ROS2 topic으로 제어
- FOLDED / EXTENDED 목표 각도 정의
- shoulder → elbow → wrist 순차 전개
- smoothstep interpolation으로 부드러운 움직임 구현

```
TOPICS = [
    '/blocking_arm/cmd_left_shoulder',
    '/blocking_arm/cmd_left_elbow',
    '/blocking_arm/cmd_left_wrist',
    '/blocking_arm/cmd_right_shoulder',
    '/blocking_arm/cmd_right_elbow',
    '/blocking_arm/cmd_right_wrist',
]

FOLDED = [0.0, 0.0, 0.0,  0.0, 0.0, 0.0]

EXTENDED = [
     math.pi / 2,   # left  shoulder: +90° → arm goes LEFT (+Y)
    -math.pi / 2,   # left  elbow   : -90° → forearm goes DOWN (-Z)
    -math.pi / 2,   # left  wrist   : -90° → hand goes RIGHT/inward (-Y)
    -math.pi / 2,   # right shoulder: -90° → arm goes RIGHT (-Y)
     math.pi / 2,   # right elbow   : +90° → forearm goes DOWN (-Z)
     math.pi / 2,   # right wrist   : +90° → hand goes LEFT/inward (+Y)
```

```
for stage_idx, pair_i in enumerate(pair_order):
    left_i, right_i = PAIRS[pair_i]

    l_start, l_end = self._current[left_i],  targets[left_i]
    r_start, r_end = self._current[right_i], targets[right_i]

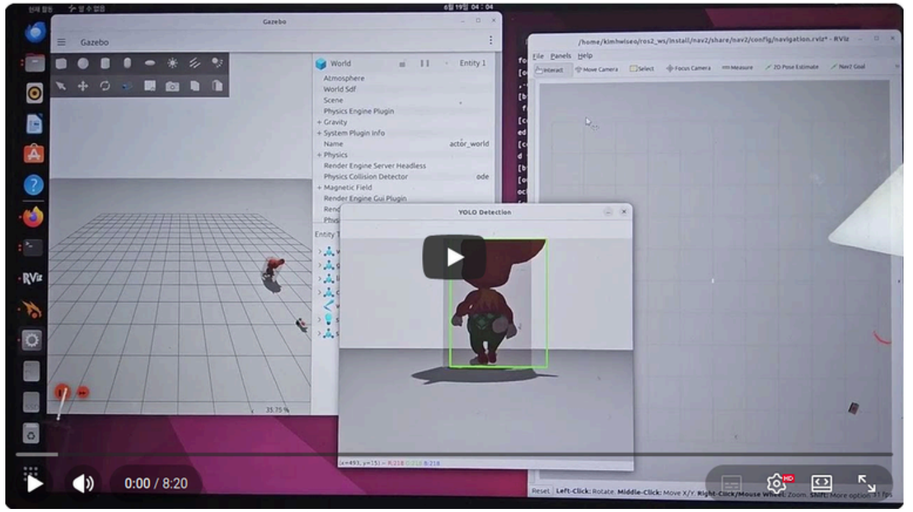
    # Smoothstep interpolation — both arms move together
    for step in range(STEPS + 1):
        t   = step / STEPS
        t_s = t * t * (3.0 - 2.0 * t)    # smoothstep
        self._send_joint(left_i,  l_start + (l_end - l_start) * t_s)
        self._send_joint(right_i, r_start + (r_end - r_start) * t_s)
        time.sleep(STEP_DT)
```

구현 결과 및 데모

영아 탐지 (YOLO)	✅ 완료	CPU 모드, 크기 기반 판단
실시간 위치 추적	✅ 완료	ActorPosePublisher 활용
최근접 콘센트 선택	✅ 완료	유클리드 거리 기반
Nav2 자율 주행	✅ 완료	RPP Controller
탐지 실패 시 탐색 회전	✅ 완료	5초 후 제자리 회전
도착 후 영아 방향 회전	✅ 완료	atan2 기반 yaw 계산
콘센트 시각화 (RViz)	✅ 완료	Marker 토픽

통합 미완 기능: blocking_arm
https://youtu.be/EJggRxzyxpA

데모 영상:



https://youtu.be/9_15_NOpqW4

스크린녹화로 촬영시 CPU의 성능이 크게 떨어지며
원래 성능이 나오지 않아 부득이하게 다른 기기로 촬영하였습니다.
actor와 로봇의 속도가 느려 발표에서는 2배속으로 재생하겠습니다.

어려웠던 점

WSL OpenGL 렌더링 문제

Windows WSL2에서 Gazebo GUI가 OpenGL 버전 미지원으로 실행 불가.
VcXsrv, MobaXterm 시도 후 실패.

→ Ubuntu 듀얼부팅으로 해결

AMCL 로컬라이제이션 불안정

빈 맵 환경에서 LiDAR 데이터 부족으로 AMCL이 로봇 위치를 잘못 추정.
RViz에서 순간이동 현상 발생.

→ Odometry 기반 로컬라이제이션으로 전환

Nav2 Controller Abort

RPP Controller가 경로 추종 중 반복 abort. transform_tolerance,
lookahead_dist 파라미터 불일치.

→ 파라미터 튜닝 및 BT transform_tolerance 조정

로봇 속도 한계

속도를 높이면 회전 불안정, 낮추면 actor보다 느림.
시뮬레이션 로봇 물리 특성의 한계.

→ actor 속도를 0.3 m/s로 낮추어 완화

개선방향

Blocking Arm 구현

Revolute 관절 판넬 링크 설계.
콘센트 도착 시 자동으로 팔을 펼쳐 물리적 차단.
시각적 억제 효과로 영아 접근 방지.

YOLO 탐지 개선

현재: 화면에서 가장 큰 물체 = 영아 (오탐 가능).
개선: 영아 전용 커스텀 모델 학습 또는 ByteTrack 객체 추적 연동.

예측 기반 콘센트 선택

영아 이동 속도 벡터 계산 → 미래 위치 예측으로 콘센트 사전 선택.
로봇이 영아보다 먼저 콘센트에 도달 가능.

실제 로봇 적용

시뮬레이션 → 실제 하드웨어 전환.
실내 환경 맵 사전 구축 후 AMCL 적용. 실제 카메라·LiDAR 센서 연동.

감사합니다